AFGNN: API Misuse Detection using Graph Neural Networks and Clustering

Ponnampalam Pirapuraj
IIT Hyderabad
Hyderabad, Telangana, India
cs23resch16001@iith.ac.in

Tamal Mondal
Oracle
Hyderabad, Telangana, India
tamalmondal495@gmail.com

Sharanya Shathavelli
Yokogawa Electric
Tokyo, Tokyo, Japan
sharanyagupta@alumni.iith.ac.in

Akash Lal
Microsoft Research
Bangalore, Karnataka, India
akashl@microsoft.com

Somak Aditya
IIT Kharagpur
Kharagpur, West Bengal, India
aditya.somak@gmail.com

Jyothi Vedurada
IIT Hyderabad
Hyderabad, Telangana, India
jyothiv@cse.iith.ac.in

Abstract

Application Programming Interfaces (APIs) are crucial to software development, enabling integration of existing systems with new applications by reusing tried and tested code, saving development time and increasing software safety. In particular, the Java standard library APIs, along with numerous third-party APIs, are extensively utilized in the development of enterprise application software. However, their misuse remains a significant source of bugs and vulnerabilities. Furthermore, due to the limited examples in the official API documentation, developers often rely on online portals and generative AI models to learn unfamiliar APIs, but using such examples may introduce unintentional errors in the software. In this paper, we present AFGNN, a novel Graph Neural Network (GNN)-based framework for efficiently detecting API misuses in Java code. AFGNN uses a novel API Flow Graph (AFG) representation that captures the API execution sequence, data, and control flow information present in the code to model the API usage patterns. AFGNN uses self-supervised pre-training with AFG representation to effectively compute the embeddings for unknown API usage examples and cluster them to identify different usage patterns. Experiments on popular API usage datasets show that AFGNN significantly outperforms state-of-the-art small language models and API misuse detectors.

CCS Concepts

• **Software and its engineering** → **Software notations and tools**; **Software verification and validation**; • **Computing methodologies** → *Machine learning*.

Keywords

API Misuse, API Usage, API Misuse Detection, API Flow Graph, Graph Neural Network (GNN), Clustering

ACM Reference Format:
Ponnampalam Pirapuraj, Tamal Mondal, Sharanya Shathavelli, Akash Lal, Somak Aditya, and Jyothi Vedurada. 2026. AFGNN: API Misuse Detection using Graph Neural Networks and Clustering. In *23rd International Conference on Mining Software Repositories (MSR '26), April 13–14, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3793302.3793372

This work is licensed under a Creative Commons Attribution 4.0 International License.
MSR '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2474-9/2026/04
https://doi.org/10.1145/3793302.3793372

1 Introduction

Developers are often unsure how to use certain library APIs (Application Programming Interfaces), even when documentation is available [44]. To find API usage examples, they commonly use online portals [7] or generative AI tools [8]. Using these examples may introduce unintentional errors and vulnerabilities to the software due to the prevalence and severity of API misuse in them [56, 57]. For example, the code sample in Figure 1, retrieved from the commit history of a GitHub project [3], demonstrates the use of the `BufferedReader.readLine()` API method at Line 3. A correct usage is shown where the `BufferedReader` object `br` is properly closed using `br.close()` at Line 7, ensuring proper resource management. In contrast, the misuse omits this close operation, potentially leading to a resource leak vulnerability. These actual examples of correct and incorrect `BufferedReader` API usage from GitHub commits highlight the occurrence of API misuse in real-world code and the importance of ensuring API usage correctness.

To this end, recent efforts have proposed tools and techniques [22, 31, 35, 53] that can analyze and detect potential API misuse or suggest correct API usage examples. These tools help minimize errors, improve security, and reduce development time, but they are limited in their ability to handle long code segments, to be computationally efficient, and to maintain accuracy. For example, CodeKernel [22], a graph kernel based approach, works well with API usage programs that are relatively short, but its accuracy drops significantly for longer code segments.

It is also possible to classify API misuse using very large language models (LLMs) such as GPT-4 [8], and DeepSeek [13] which offer strong reasoning capabilities for code understanding, but their high computational cost, hard to fine-tune nature limit their practical applicability in API-misuse detection. This motivates the development of lightweight, structure-aware approaches, such as small language models (LMs) [1] and graph-based models, that can efficiently learn API-usage patterns without relying on massive compute or costly

[1] Here, by "small LMs", we refer to smaller models (Million-level parameters, e.g. 125M) like GraphCodeBERT [24] and UnixCoder [23], while by "LLMs", we refer to models (Billion-level parameters, e.g. 175B) like GPT [8].

```java
public static void main(String[] args) throws Exception {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    StringTokenizer st = new StringTokenizer(br.readLine());
    StringBuilder sb = new StringBuilder();
    ...
    for (int i = 1; i <= N; i++) {
        sb.append(count[i]).append(" ");
    }
    System.out.println(sb.toString());
    /* BufferedReader is not closed */
}
```

(a) API misuse example of `BufferedReader.readLine()`

```java
public static void main(String[] args) ... {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    StringTokenizer st = new StringTokenizer(br.readLine());
    StringBuilder sb = new StringBuilder();
    ... //for loop not shown
    System.out.println(sb.toString());
    br.close();
}
```

(b) Correct usage example of `BufferedReader.readLine()`

Figure 1: Usage of `BufferedReader.readLine()` **API.**

prompting. Embeddings generated by small LMs can be used to detect API misuse. However, small LMs such as CodeT5+ [51], CodeBERT [18], GraphCodeBERT [24] and UniXcoder [23] even though good at providing task-agnostic embeddings, perform poorly in detecting API misuse because: (1) they do not take into account API-specific data and control-flow relationships but instead focus on the sequence of code tokens, (2) they suffer information loss for longer code snippets due to input length limitation. Further, Graph Neural Network (GNN) based models like GraphCode2Vec [36] require expensive static analysis on the intermediate representation of the code to obtain the embeddings.

To address these challenges, we propose AFGNN, a lightweight GNN-based framework that detects potential API misuse in input code efficiently. AFGNN generates embeddings to capture API usage and then clusters the embeddings. To model the API usage patterns, AFGNN uses a novel API Flow Graph (AFG) representation that captures the data flow, control flow, and API call sequence in the code that flows through the API call site. AFGNN uses self-supervised pre-training with AFG representation to effectively compute the embeddings for unknown API usage examples. These embeddings are then clustered based on their similarity to identify the usage patterns. A larger cluster indicates frequent API usage, suggesting correct use, while a smaller cluster denotes infrequent usage, indicating potential misuse. Prior work [22, 27, 32, 33, 39] has shown that frequent patterns in real-world projects often correspond to correct API usage.

Our pre-trained GNN model is a fraction of existing small LMs in terms of model size and so needs less memory and computation power. Further, our evaluation shows that AFGNN has superior performance in understanding API usage and detecting API misuse compared to state-of-the-art small LMs and misuse detectors.

The key contributions of this work are as follows:

- A manually labelled API usage dataset with widely used Java APIs suitable for clustering API usage patterns.
- A new API Flow Graph (AFG) representation that captures the data flow, control flow, and API call sequences in a code snippet.
- AFGNN, a GNN-based model that uses AFG representation and self-supervised pre-training to learn flow-aware code embeddings, and clusters them to discover API usage patterns.
- Extensive evaluation showing the effectiveness of our approach over state-of-the-art small LMs in API-usage clustering and misuse detection, along with comparison to existing misuse detectors on real-world examples. Ablation study assessing the impact of different AFGNN components and validating our design choices.

2 Related Work and Background

Language Models and GNNs for Code Understanding. Pre-trained language models such as CodeBERT [18], GraphCodeBERT [24], UniXCoder [23], and CodeT5+ [51] are relatively small in size (million-parameter scale) and have shown strong performance in code understanding tasks. These encoder or encoder-decoder models process code syntax and semantics to generate embeddings that can be used to detect API misuse but they struggle with longer code snippets due to input length constraints. GraphCodeBERT enhances embeddings via data-flow graphs, while UniXCoder integrates abstract syntax trees and comments. However, the absence of control-flow modeling in GraphCodeBERT limits its effectiveness in capturing complex API usage patterns. CodeT5+ leverages the T5 architecture with advanced objectives for good performance.

Since program structures are inherently graph-based, with representations like Abstract Syntax Tree (AST), Control-Flow Graph (CFG), Data-Flow Graph (DFG) [17], and Program Dependence Graph (PDG) [19], Graph Neural Networks are well-suited for program analysis, having demonstrated remarkable success in learning representations that capture both local and global graph structures. Different types of GNNs include Graph Convolutional Networks (GCNs) [28], Graph Attention Networks (GATs) [47], Relational Graph Convolutional Networks (RGCNs) [46], and Relational Graph Attention Networks (RGATs) [14]. GCNs extend the concept of convolution from grids to graphs, aggregating information from neighbouring nodes. GATs enhance this by applying attention mechanisms to weigh the importance of neighbouring nodes differently. RGCNs handle multi-relational data by incorporating edge types into the aggregation process, while RGATs combine relational information with attention mechanisms. GNNs iteratively update node representations through message passing, aggregating information from neighbour nodes, combining it with their own features, and updating their state. Key strategies in GNNs include the choice of aggregation functions, which can be mean, sum, or max operations, and the use of normalization techniques to stabilize training. A broader discussion on adapting GNNs for code representation can be found in the study by Allamanis et al. [9]. As we represent input

Figure 2: Overview of AFGNN. Given method-level code snippets, we construct API Flow Graphs (AFGs), generate their embeddings using AFGNN, and cluster the embeddings to identify frequent usage patterns and potential misuses.

code as a graph (AFG), AFGNN uses GNN architecture to leverage GNNs' efficiency in understanding such graph structures.

Luo et al. [34] use Compressed Abstract Graph (CAG), a compact representation of AST, to represent code for vulnerability detection that preserves key structure and semantics while accelerating model training. They used MPNet [45] to get node embeddings, followed by a two-layer GNN with soft attention pooling. While they emphasize compressed graphs for vulnerability detection, AFGNN introduces AFGs by modeling data and control dependencies and API sequences for API Misuse Detection. Zhou et al. [49] extract task-specific subgraphs from Code Property Graphs (CPGs) [54] that integrate ASTs, control flow graphs, and program dependency graphs to capture comprehensive code semantics. Nodes are encoded via operation types, function types, and semantic vectors from Word2Vec [38], processed by a GCN with attention. In contrast, AFGNN introduces API sequence modeling in AFGs.

Graph-based Approaches for API Misuse Detection. Graph-based methods like Code-Kernel [22] and GraphCode2Vec [36] represent code as graphs for embedding. Code-Kernel uses graph kernels but lacks control flow modeling and machine learning support (heuristic-based), limiting its applicability for detecting large, complex API usage patterns. GraphCode2Vec fuses lexical and dependency embeddings using static analysis. While effective, it faces scalability challenges on large or complex codebases due to its costly static analysis. It also cannot be applied directly to our datasets, as it requires compilable Java code for inference, whereas our method-level data often contains partial, non-compilable code. ADG-Seq2Seq [35] uses API dependency graphs but omits control dependencies. In contrast, along with data dependencies, AFGNN models control flow and the API call sequences in AFGs.

Among rule-based and knowledge-driven methods, Xia Li et al. [31] conducted a large-scale study on GitHub bug-fix commits to detect specific API misuse types like guard conditions and exception handling, using fine-grained AST differencing and a lightweight intra-procedural analysis. However, their approach relies on manually crafted detection rules. MuDetect [12], represents code with API-Usage Graphs (AUGs) and applies a greedy, semantic-aware subgraph mining and specialized graph matching, along with a ranking strategy, to improve precision and recall over prior methods. Ren et al. [41] built a knowledge graph from API documentation to model usage constraints like call order and condition checks, for detecting API misuses. Li et al. [29] presented an improved API misuse detection method that integrates usage constraints from client code, documentation, and libraries to generate comprehensive AUGs. Ma et al. [37] propose GraphiMuse, which encodes API usages as AUGs and learns probabilistic usage patterns by aggregating rules from code and documentation, representing each rule's trustworthiness as a probability. Wang et al. [50] proposed APICAD, which enhances API misuse detection by inferring specifications from both code and documentation using static analysis and symbolic execution, targeting C++ programs at the LLVM IR level.

Unlike these methods, AFGNN is an unsupervised GNN model that learns API usage patterns directly from real examples. Our evaluation on the MUBench [11] dataset shows AFGNN's effectiveness over prior methods [12, 29, 37, 41] in terms of precision, recall, and F-score. Further, unlike AUGs, which capture control and data flow, AFGs additionally model API call sequences with nodes as statements (not variables), enabling more precise representation of API usage behaviour. Futhermore, as APICAD [50] does not support Java, a direct comparison with it is infeasible.

Kang and Lo [27] proposed Actively Learned Patterns (ALP), which frames API misuse detection as a classification task using active learning and human supervision to mine subgraphs that effectively distinguish correct from incorrect usages. In contrast, AFGNN employs an unsupervised GNN model to learn API usage patterns and detect misuses without manual intervention. (We could not directly compare with this work due to errors encountered while using the artifacts.)

Domain-specific API misuse detection. Recent work has explored domain-specific API misuse detection. For example, LLMAPIDet [52] detects misuses of deep learning (TensorFlow and PyTorch) APIs by applying few-shot prompting with a heavyweight LLM (ChatGPT [40]). Even on domain-specific test set, it detected only 48 out of 291 API misuses (16.49%). In contrast, AFGNN is a lightweight, semantics-aware, and language-agnostic graph-based model that learns API usage patterns directly from the AFGs, making them not directly comparable. Further, Cryptolation [21] and CryptoGo [30] detect cryptographic API misuses in Python and Go, respectively, using static analysis and manually crafted rules, highlighting the prevalence of misuse patterns in security-critical code. In contrast, AFGNN is a general, graph-aware, unsupervised approach that learns API usage patterns directly from real code, without relying on domain assumptions or handcrafted rules.

3 AFGNN Framework

We present AFGNN, a GNN-based framework that detects potential API misuse and recommends frequent ways of using an API based on open-source examples by using a new API Flow Graph

(AFG) representation. Figure 2 shows the end-to-end workflow of our framework. As shown, our framework begins by generating the AFGs using an AFG generator for the method-level API usage examples (Section 3.1). Subsequently, our GNN model generates the graph embeddings from these AFGs (Section 3.2). These graph embeddings are then clustered (Section 3.3). The hypothesis is that the large clusters indicate frequent/common API usage patterns, while smaller clusters indicate infrequent usage or potential misuse [22, 27, 32, 33, 39]. These clusters can be used for different API-related tasks, such as recommending frequent usage patterns or detecting potential API misuse.

3.1 API Flow Graph (AFG) Representation

3.1.1 AFG Definition. An API Flow Graph (AFG) is a directed graph representation of a given API usage code snippet, defined as a three-tuple $G = \langle V, E, X \rangle$. Here, V is the set of vertices, with each vertex representing an embedding of a program line in the code, E is the set of edges denoting the dependencies between program lines, and X is the set of edge labels $\{FD, CD, SE\}$, representing data flow edges, control flow edges, and sequence edges. Figure 3 shows an API usage example and its corresponding AFG with nine vertices (line numbers are shown instead of embeddings for simplicity) and 11 edges. We describe the three types of edges below.

Data flow edges (FD). A data flow edge in AFG represents a data dependency between API-related operations in the code, indicating the flow of data from a *src* node to a *target* node. For example, in Figure 3, the variable `statement` at Line 4 stores the result of an API call `createStatement()`, and `statement` is later used at Line 8 to invoke another API call `execute()`. Hence, an FD edge connects the node representing Line 4 to the node representing Line 8.

Control flow edges (CD). A control flow edge represents a control dependency between API-related operations, indicating the flow of control (execution order) from a *src* node to a *target* node. The *src* node can be the starting line of a function, a branch condition of an `if` statement, a loop condition, a `switch` condition, or the beginning of a `try-catch` block. The *target* node is a statement located within the bodies of these structural constructs. These edges model instances where an API usage is preceded by verification checks (such as null pointer or array bounds checks) or protected by exception-handling constructs (like try-catch). For example, in Figure 3, the API call `execute()` at Line 8 is executed only if `statement != null` at Line 7. Hence, a CD edge connects the node representing Line 7 to the node representing Line 8.

Sequence edges (SE). A sequence edge represents the sequential order in which API calls are made on a particular receiver object, capturing the correct sequence of operations to ensure proper API usage within a code snippet. SE edges are created automatically by linking each API call to its immediately following API call on the same object. For example, in Figure 3, `connection.createStatement()` at Line 4 is called before `connection.close()` at Line 11, and both calls operate on the same `connection` object. Hence, the SE edge connects the node with Line 4 to the node with Line 11, but not Line 3 as the latter invokes an API on a different object (`DriverManager`). Similarly, in Figure 1, a sequence edge connects the `br.readLine()` and `br.close()` calls to indicate the necessary order of execution, where the closing of the `BufferedReader` (`br.close()`) must follow the reading of a line (`br.readLine()`), reflecting common practice, where resources are closed after use.

3.1.2 AFG Generator. We implement an AST-based AFG generator for Java that constructs AFGs by capturing data flow, control flow, and API call sequences at the source-code level. Our AST-based approach offers the advantage that it does not require compilable Java code, but it can generate AFG from any syntactically correct (partial) Java code, which allows us to consider any code snippet that contains API usage. AFG generator parses each Java code snippet into an AST and extracts statement-level nodes (e.g., variable declarations, assignments, API calls, conditionals). Using the generated AST, FD edges are added by linking source nodes representing variable definitions to target nodes representing their corresponding uses. CD edges are added from source nodes corresponding to branch conditions in structural constructs (e.g., conditionals, loops, and `try-catch` blocks) to target nodes representing statements within their bodies. SE edges are created by linking each API call to its immediate successor API call on the same receiver object. To construct these edges, we identify API callsites (location where an API is invoked, e.g., `statement.execute(sql)` is a callsite for `execute()` API) and then analyze the surrounding statements to derive object-specific call sequences.

Figure 3 shows the generated AFG for a sample API usage example. In reality, every node in an AFG is an embedding of a program line in the code snippet. We generate these embeddings by using CodeT5+ [51] in our approach (see Section 7).

3.1.3 AFG Pruning. At inference time, given a targeted API of interest, a complete AFG is unnecessary as only the portion relevant to the API's usage is required. While the raw AFG (discussed above) captures the data flow, control flow, and API call sequence across code lines, it often includes extraneous nodes and edges that are unrelated to the specific usage of the targeted API, particularly in snippets containing multiple APIs. To address this, we apply a pruning algorithm on the raw AFG that retains only the subgraph relevant to the target API call. Specifically, it preserves nodes that are predecessors or successors of the API node, defined as those reachable from or to the API call through control, data, or sequence edges. The pruning algorithm further eliminates non-informative structures, such as class-level links, CD edges originating from method signatures, and self-loops, while merging nodes mapped to the same code line for compactness. The resulting subgraph compactly captures the target API's contextual usage. We further tested reversing or duplicating edges, but they showed no improvement and are disabled by default.

In Figure 3, if the target API method is `Statement.execute()`, then Line 6 and Line 11 are not relevant to API usage because Line 6 is only used for logging, and Line 11 closes the connection object, and they are not connected to the `Statement.execute()` call (node 8). While `createStatement()` at Line 4 and `close()` at Line 11 are relevant for analyzing the usage of the `Connection.close()` API method, it is outside the scope when `Statement.execute()` is the targeted API under investigation. Since our focus is on detecting misuse patterns for a specific API, we consider one API misuse at a time during inference, even in real-world scenarios where multiple APIs appear within a snippet. Therefore, as the nodes numbered 6

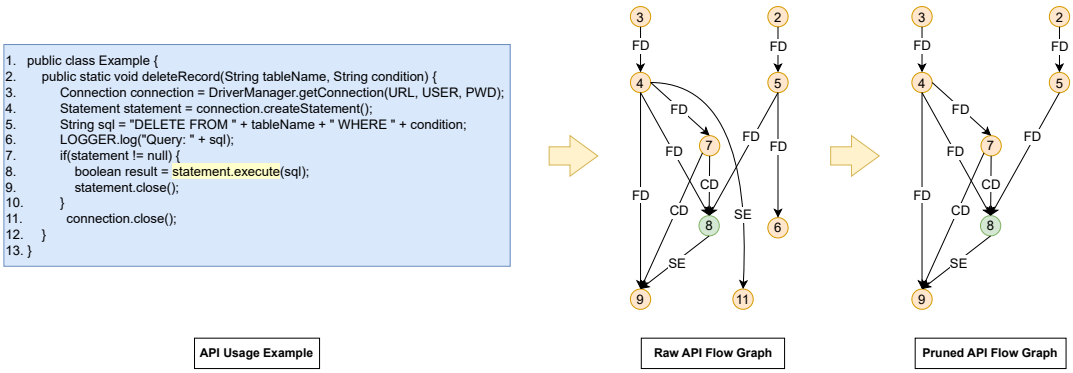

Figure 3: Example of an API Flow Graph (AFG) for a `Statement.execute()` usage. The graph captures data flow (FD), control flow (CD), and sequence (SE) edges among program lines.

and 11 are not reachable to/from the API call node (node 8), they are removed from the final AFG as part of the pruning process.

We evaluated AFG pruning on a dataset containing multiple APIs alongside the targeted API, and found it effective in multi-API settings. This is because it removes nodes and edges related to non-target APIs while preserving only statements that influence the targeted API call, thereby reducing noise from unrelated API usage. This design naturally scales to complex code contexts common in real-world scenarios, where its benefits in focusing on the targeted API are expected to be even greater. We apply the AFG pruning process only during evaluation/inference, but not during the pre-training of AFGNN, as it requires knowing the target API in advance and is thus impractical for the pre-training stage. During pretraining, AFGNN instead uses a generic context prediction objective on the raw AFG.

3.2 AFGNN Model

Graph Neural Networks (GNN) are widely used to obtain meaningful representations of nodes or entire graphs. We have used GNN to get the embedding from the API Flow Graph (AFG) representation. This section explains how we pre-train our AFGNN model.

3.2.1 AFGNN Architecture. AFGNN can support standard Graph Neural Network (GNN) architectures, and we empirically identified the most effective GNN architecture for API misuse detection. AFGNN uses the AFG representation of code and learns the node representations through message-passing, where nodes iteratively exchange and aggregate information from their neighbours. A k-layer GNN can aggregate the node embeddings up to k-hops, with the message-passing function defined by its architecture. For AFGNN, we have experimented with four types of GNN architectures: GCN [28], GAT [47], RGCN [46], and RGAT [14] (explained in Section 2). We present in this paper the results using GCN and RGCN architectures in the AFGNN model due to their superior performance over GAT and RGAT. A five-layer GNN architecture is sufficient for capturing the complexity of our AFGs because the pruned graphs in our dataset are small, with edges ranging from 1 to 61, allowing information to propagate across all nodes reachable from the API callsite.

After the message passing step, we obtain the final set of node embeddings for the input AFG. Standard pooling functions like mean, max, and min, though common, are not optimal for capturing API flow information, as they lose API-specific details (based on our experiments). Using the embedding of the API callsite (API node) as the graph representation yielded better results because message passing centering the API call captures its neighbourhood with API-related operations. Hence, we present the results using the AFGNN embedding of the API node. For code examples with multiple callsites having the same API call, we apply mean pooling to the API nodes' embeddings to obtain the final API flow embedding.

3.2.2 AFGNN Pre-training Dataset. Pre-training trains a model on a large dataset in an unsupervised or self-supervised way to learn general code patterns, and we use the pre-trained AFGNN model later for inference and generating graph embeddings, without fine-tuning. Since our downstream task is unsupervised clustering, effective API-usage clustering through transfer learning requires pre-training on a large and relevant dataset.

For AFGNN, we pre-train on the java-large [5] dataset from Code2Seq [10], sourced from 9, 500 top-starred GitHub Java projects created since January 2007 that have active software development histories. We extracted method-level code examples from the raw project-level data, focusing only on those that use popular Java library APIs for our downstream API-usage clustering tasks. We focus on popular Java APIs because they are widely used in real-world software projects, enabling the model to learn representative and generalizable API usage patterns, consistent with prior empirical studies [12, 29, 37, 41] on the MuBench benchmark. In total, we collected approx. 1.5 million unique Java API usage examples that are split into training, testing, and validation data sets in an 8:1:1 ratio. For pretraining, we considered AFGs with at least three edges to ensure informative graph structures and support effective and unbiased model learning.

Our pre-training dataset includes diverse code samples from various domains (identified through manual inspection) such as Networking, Concurrency and Multithreading, Database Operations, File and I/O Operations, Date and Time Management, Resource Management and Class Loading, String and URI Handling, Graphics, System and Memory Management, and GUI Operations. This

Figure 4: AFGNN pre-training pipeline

Input: E: Embeddings as $[e_1, e_2, \ldots, e_n]$, where e_i is the AFGNN embedding for ith example; M: Birch clustering algorithm
Output: C: Cluster labels as $[c_1, c_2, \ldots, c_n]$, where c_j is the predicted cluster label for jth example

```
Procedure clusterTheEmbeddings(E, M)
    min_db_score ← ∞;
    C ← [0, 0, ..., 0] ;    // Initially, all examples are in the 0th cluster
    for cluster_cnt ← 2 to n do
        pred_labels ← M(E, cluster_cnt);
        db_score ← calcDBscore(E, pred_labels);
        if db_score < min_db_score then
            min_db_score ← db_score;
            C ← pred_labels ;    // Clustering improved
```

Figure 6: Algorithm to find the best clustering

broad exposure helps AFGNN capture diverse API usage patterns, enhancing its ability to generalize, and accurately evaluate API usage.

3.2.3 AFGNN Pre-training. For pre-training, we first generate AFGs from the Java method-level examples. Since GNNs require dense node representations as input, we can initialize the nodes with embeddings from small LMs such as CodeT5+ [51], CodeBERT [18], and UniXcoder [23]. We use CodeT5+ for initialization, as it provides the best overall performance. Our pre-training experiment on the context-prediction task show that CodeT5+ achieves an accuracy of 0.902, outperforming UniXcoder (0.884) and CodeBERT (0.786), which is why we select CodeT5+ to initialize node representations.

Figure 4 illustrates the pre-training pipeline of AFGNN. We pre-train AFGNN using the context prediction objective [25], where subgraphs are used to predict surrounding graph structures, aiming to map nodes in similar structural contexts to nearby embeddings. This objective naturally aligns with our goal to get meaningful embeddings for API nodes, ensuring that API nodes of similar API usage examples have similar embeddings as they would appear in a similar context. We have not pruned the AFGs during pre-training as context-prediction pre-training is unsupervised and not specific to API usage. Thus, AFGNN leverages unsupervised context-prediction pretraining to generate flow-sensitive code embeddings, which are then clustered (see Figure 5) to identify API usage patterns and detect potential misuse, enabling independence from labeled data and offering flexibility in analyzing unfamiliar API usages.

3.3 API Usage Clustering

We use the pre-trained AFGNN to generate embeddings for API usage examples and then cluster them using the popular BIRCH

Figure 5: AFGNN clustering pipeline

algorithm [55] to identify API misuses (as shown in Figure 5). We chose BIRCH due to its hierarchical and memory-efficient design, incremental clustering capability, and practical speed advantages, as well as its flexibility in fine-tuning via the threshold parameter. Although our evaluation dataset is relatively small, BIRCH demonstrated competitive performance and interpretability compared to alternative clustering algorithms reported in the literature. We report results using the optimal thresholds for AFGNN and baseline models for the API usage clustering.

The optimal number of clusters depends on factors such as the specific API and example count. To determine the best clustering, we use the Davies-Bouldin (DB) [16] metric, selecting the number of clusters that yield the best DB score, with lower scores indicating better clustering.

$$DB = \frac{1}{K} \sum_{i=1}^{K} \max_{j \neq i} \left(\frac{S_i + S_j}{M_{ij}} \right) \quad (1)$$

where K is the number of clusters, S_i, and S_j denote the average distances of points in clusters i, and j to their respective centroids, M_{ij} is the distance between cluster centroids of clusters i and j. The algorithm in Figure 6 is used to determine the best clustering for a particular API. Initially, all the examples are considered to be in a single cluster, and iteratively clustering the examples for all possible cluster counts (from 2 to the total number of examples, n), calculating the DB score in each iteration and selecting the final clustering with the minimum DB score.

4 Experimental Methodology

We evaluate AFGNN on the following research questions.
RQ1. How effective are the AFGNN model embeddings at clustering different API usage patterns?
RQ2. How effectively does AFGNN detect API misuse in real-world examples (written by developers) from MUBench [11] dataset?
RQ3. How do addition of sequence edges help AFGNN in identifying API usage patterns?
RQ4. How does AFG pruning (removing nodes and edges not related to the target API usage) affect AFGNN performance?

Experimental setup. We conducted all our experiments on a machine with an Intel Xeon Gold 5326 CPU (32 cores, 2.90 GHz) and a 1.41 GHz NVIDIA Ampere A100 GPU with 80 GB global memory, which was used for training and inference of AFGNN. We trained the AFGNN model with the context-prediction pre-training objective, using a learning rate of $5e^{-5}$, a batch size of 256, and the Adam optimizer ($\epsilon = 1e^{-8}$) until validation accuracy plateaued for five consecutive epochs. We implemented the AFGNN pipeline in Python (v3.12.2) with PyTorch (v2.2.1). We present the performance results of two variants of AFGNN: 5-layer GCN and RGCN (denoted as AFGCN and AFRGCN), both pre-trained with the same setup.

As baselines, we consider UnixCoder and GraphCodeBERT, two state-of-the-art small LMs (criteria explained in Section 7), and have utilized the same system for inference. For all the baseline models, we used pre-trained weights from HuggingFace [4]. Since very large LLMs (e.g., GPT-4, DeepSeek) are computationally expensive and hard to fine-tune, we restrict our baselines to lightweight small LMs and graph-based approaches that can be trained and evaluated efficiently within our setup. Further, a comparison with CodeKernel [22] is not feasible as it is not open-source, and we contacted the authors, who confirmed that the code is currently unavailable. All our datasets and source code are publicly available [1].

Metrics Used. To determine the optimal number of clusters for a particular API and the clustering quality, we use an intrinsic evaluation measure Davies-Bouldin [16] metric (see Section 3.3). To compare the clusters produced by AFGNN with those from the baselines, we used two popular external clustering index metrics: Rand Index (RI) [26] and Mutual Information (MI) [48], which measure the similarity between the predicted clusters and ground truth. We use the "Adjusted" versions of these metrics from scikit-learn library [6], as they penalize random labeling and large numbers of clusters. All clustering metrics are sourced from the scikit-learn library [6]. The RI (eq. 2) measures the proportion of sample pairs that are either assigned to the same cluster in both the predicted and ground truth labels, or assigned to different clusters in both.

$$RI = \frac{\text{Number of agreements}}{\text{Total number of pairs}} \tag{2}$$

The Adjusted Rand Index (ARI) measures clustering similarity by adjusting the observed agreement between two clusterings to account for agreement expected by chance. ARI ranges from -1 (poor agreement) to 1 (perfect match), with 0 indicating random labeling.

The MI measures the amount of shared information between two clusterings. It is given by:

$$MI(U,V) = \sum_{u \in U} \sum_{v \in V} P(u,v) \log \left(\frac{P(u,v)}{P(u)P(v)} \right) \tag{3}$$

where $P(u)$ and $P(v)$ are the probabilities of clusters u and v in U (the true labels) and V (the predicted labels), respectively, and $P(u,v)$ is their joint probability.

Adjusted Mutual Information (AMI) normalizes mutual information by correcting for the agreement expected by chance, using the entropies of the two clusterings, and ranges from 0 (no agreement beyond chance) to 1 (perfect agreement).

5 Dataset

This section describes the two datasets used in our evaluation: (1) a new labeled dataset based on CodeKernel [22] data for API usage clustering evaluation (RQ1) and ablation studies (RQ3 and RQ4), and (2) the API misuse dataset from MUBench for comparison with baselines (RQ2), respectively.

5.1 API Usage Clustering

Due to the absence of real-world expert-annotated API usage clusters, we curate a new dataset based on CodeKernel [22], which provides usage examples for a set of 25 popular Java APIs via its GitHub page [2], with the number of examples per API ranging from 6 to 1, 368. Among them, we consider a total of 21 APIs (listed in Table 2) that had at least 30 examples. For each selected API, we randomly sampled and manually labeled around 30 examples (excluding very small snippets to ensure meaningful clustering) to keep human annotation feasible, maintain consistency across APIs and avoid possible dataset imbalance.

Table 1: Sample rules for manual API usage clustering

Rule	Description
R1	API calls with different signatures (distinct argument types due to inheritance/polymorphism) result in separate clusters.
R4	API usage enclosed within `if-else` or `try-catch` blocks leads to different clusters.
R5	If the result of an API call is assigned to, or appended to a variable (e.g., via append() or add()), it belongs to a separate cluster.
R6	Multiple calls to the API within loops or conditional statements should be clustered differently.
R10	API calls within nested conditionals or nested loops should form separate clusters.

```
getResourceStreamWithClassLoader(ClassLoader
      classLoader,String path){
    if (classLoader != null) {
      URL url = classLoader.getResource(path);
      if (url != null) {
        return new UrlResourceStream(url);
      }
    }}
```

(a) API result is assigned to a variable

```
findCurrentResourceVersion(String resourceUrl){
        ClassLoader cl =
             getClass().getClassLoader();
        return cl.getResource(resourceUrl);
}
```

(b) API result is directly returned without assignment

Figure 7: Examples for Rule R5. Although both examples call `getResource()`, they differ in how the result is used.

To generate high-quality labels, we enlisted two Computer Science graduate students, each with over two years of Java development experience. To minimize subjective bias, they first independently reviewed a subset of examples and collaborated to derive a unified set of 14 rules for labeling the dataset. They first annotated a subset of examples separately, compared their labels, and refined the rules iteratively to resolve disagreements. After reaching consensus, the annotators re-labeled the dataset using the finalized rules, resulting in a consistent and reliable ground-truth dataset.

We show 5 representative rules in Table 1 (among 14, due to space limitations). The rules cover factors influencing clustering decisions, such as variations in API signatures, parameter scopes, control-flow contexts, and other usage patterns. As illustrated in Figure 7, although both examples invoke the same API `getResource()`, they differ in how the result of the API call is handled. In the first example, the result is assigned to a variable for further checking, while in the second example, the API result is returned directly without any intermediate handling. As per Rule R5, such semantic differences lead to separate clusters, as they reflect distinct API usage patterns. The full rule set is in the supplementary material (Appendix A and B) [1]. Using these rules, annotators independently clustered

```
void pattern(Map foregroundDomainMarkers, ..., Marker marker) {
  ArrayList markers = (ArrayList) foregroundDomainMarkers.get(...);
  if (markers != null) {
    markers.remove(marker);
  }
}
```

```
Line_1 $$ void pattern(...) { --> Line_2 $$ ArrayList markers = ... [FD]
Line_2 $$ ArrayList markers = ... --> Line_3 $$ if (markers != null) { [FD]
Line_3 $$ if (markers != null) { --> Line_4 $$ markers.remove(marker); [CD]
Line_2 $$ ArrayList markers = ... --> Line_4 $$ markers.remove(marker); [FD]
Line_1 $$ void pattern(...) { --> Line_4 $$ markers.remove(marker); [FD]
```

(a) Correct use example from MUBench with its AFG.

```
void pattern(Map foregroundDomainMarkers, ..., Marker marker) {
  ArrayList markers = (ArrayList)
        foregroundDomainMarkers.get(...);
  markers.remove(marker);
}
```

```
Line_1 $$ void pattern(...) { --> Line_2 $$ ArrayList markers = ... [FD]
Line_1 $$ void pattern(...) { --> Line_3 $$ markers.remove(marker); [CD]
Line_1 $$ void pattern(...) { --> Line_3 $$ markers.remove(marker); [FD]
Line_2 $$ ArrayList markers = ... --> Line_3 $$ markers.remove(marker); [FD]
```

(b) Misuse example from MUBench with its AFG.

Figure 8: Examples of correct use and misuse from MUBench, shown with their corresponding AFGs.

the examples. Further, they cross-verified each other's annotations and resolved disagreements using mutual discussions. This led to a consistent set of labelling rules, which can be extended to other APIs. The curated dataset was used solely for testing, not training, in our RQ1 evaluation.

5.2 MUBench dataset

MUBench [11] is a popular dataset for API misuse detection, comprising 162 correct usage examples and corresponding misuse descriptions from 67 open-source Java projects. Since it is built from real-world open-source Java projects, MUBench offers a reliable and practical dataset for evaluating API misuse detection. The dataset includes correct usage examples, supplemented by YML files detailing misuse cases, file URLs, descriptions, and proposed fixes. Due to the dataset's age, some repositories were inaccessible, so we collected all available examples and YML files and reconstructed the missing misuse cases based on the YML descriptions. Further, since AFGNN operates at the method level, we focused on extracting only the methods associated with API misuse. As a whole, we compiled a total of 324 examples: 162 correct usage examples downloaded directly from the MUBench repository and 162 corresponding misuse examples. Figure 8 shows examples of correct use and misuse from MUBench alongside their corresponding AFGs. In the correct case, the conditional check `(if (markers != null))` is preserved in the AFG, ensuring safe API usage. In contrast, the misuse omits this check, and the resulting AFG lacks the control-dependency edge, highlighting how AFGs capture semantic differences between correct and incorrect API usages.

6 Results

6.1 RQ1: Effectiveness of AFGNN

AFGNN embeddings are evaluated for their effectiveness in clustering similar API usages, using labeled examples from 21 popular Java APIs (see Section 5.1). Table 2 presents the clustering results for four different models (AFGCN, AFRGCN, and the two baseline models), showing the Rand Index (RI) and Mutual Information (MI) scores based on predicted clusters and ground-truth. In each row, the highest RI and MI scores are indicated in bold. The suffix "Birch (1.5)" in the first row indicates the threshold of 1.5 used in the BIRCH clustering algorithm to control the maximum diameter of subclusters (similarly, 2.5).

```
public void paint(Graphics2D g2d){
    g2d.setColor(new Color(96, 96, 96));
    g2d.fill(area);
}
```

Figure 9: A short usage example of `Graphics2D.fill()` API.

The last row of Table 2 reports the average RI and MI scores, where AFRGCN outperforms all models. The closest is GraphCodeBERT, which is nearly 95 times larger than AFRGCN. Thus, AFGNN is not only more effective than state-of-the-art small LMs, but it is also more efficient and less resource-intensive. GraphCodeBERT benefits from data flow modeling, but it still lags behind AFGNN, which additionally captures API-centric control flow and call sequences. Some APIs have very short examples, where token-based models such as UnixCoder and GraphCodeBERT perform well. Figure 9 illustrates such a case for the `Graphics2D.fill()` API. While AFGNN relies on graph structure, the corresponding AFG in this setting contains only a few edges, thereby limiting contextual information. Nevertheless, p-value analysis (discussed next) confirms that AFGNN's performance gains are statistically significant, and its 100× smaller size highlights its practical value. UnixCoder has performed the best for three APIs `Graphics2D.fill()`, `Timestamp.compareTo()` and `ExecutorService.submit()` where the code examples are typically small (2-3 lines in some cases) and textual patterns matter more than the data and control flow.

P-Value evaluation. AFGNN performs well in most API clusterings, as indicated by the RI and MI scores in Table 2. However, some scores for GraphCodeBERT and AFGNN, as well as UnixCoder and AFGNN, are quite close, indicating less difference between them. To understand the statistical significance of these results, we conducted a p-value evaluation, using pairwise comparisons and employing a significance threshold of 0.05, and the results are summarized in Table 3. Given the multiple pairwise statistical tests, we applied the Benjamini-Hochberg (BH) correction to control the Type-I error rate, with the adjusted p-values shown in the fourth and fifth columns of Table 3. For our evaluation, the null hypothesis stated that there is no variation in the models' performances.

Significant differences exist between UnixCoder and AFRGCN (raw p-values of 0.0022 for RI and 0.0008 for MI, with BH-corrected p-values of 0.0067 and 0.0032, respectively) as well as between GraphCodeBERT and AFRGCN (raw p-values of 0.0104 for RI and 0.0227 for MI, with BH-corrected p-values of 0.0179 and 0.0302).

Table 2: External index results for API usage clustering. The upward arrow indicates that higher RI and MI scores are better. The highest MI scores and RI scores are highlighted in each row

API methods	UnixCoder-Birch (1.5)		GraphCodeBERT-Birch (1.5)		AFGCN-Birch (1.5)		AFRGCN-Birch (2.1)	
	126M		124M		331K		1.3M	
	RI score ↑	MI score ↑	RI score ↑	MI score ↑	RI score ↑	MI score ↑	RI score ↑	MI score ↑
Classloader.getResource()	-0.004	-0.005	**0.401**	**0.454**	0.254	0.23	0.322	0.396
Thread.start()	-0.003	-0.003	0.158	0.176	0.114	0.231	**0.439**	**0.472**
Statement.execute()	0.219	0.235	0.237	0.33	**0.34**	**0.422**	0.174	0.184
BufferedReader.read()	0.027	0.055	0.299	0.366	0.144	0.238	**0.412**	**0.492**
Timestamp.compareTo()	**0.497**	**0.497**	0.239	0.239	0.39	0.489	0.325	0.325
DataInputStream.readLine()	0.163	0.182	0.4	0.416	0.383	0.423	**0.41**	**0.465**
ServerSocket.bind()	0.114	0.141	**0.288**	**0.34**	0.098	0.198	0.231	0.298
ExecutorService.submit()	**0.665**	**0.665**	0.192	0.205	0.021	0.056	0.278	0.306
URI.getFragment()	0.302	0.302	0.449	0.538	0.433	0.449	**0.683**	**0.719**
Calendar.getTime()	0.219	0.235	0.304	0.423	0.064	0.207	**0.313**	**0.425**
Socket.connect()	0.302	0.335	**0.608**	**0.671**	0.198	0.412	0.58	0.621
JPanel.add()	0.114	0.157	0.273	0.352	0.055	0.122	**0.384**	**0.49**
FileChannel.read()	-0.004	-0.004	**0.171**	**0.182**	-0.007	-0.007	0.142	0.157
DateFormat.format()	-0.004	-0.004	0.141	0.189	0.042	0.096	**0.364**	**0.339**
ClassLoader.loadClass()	-0.004	-0.004	0.087	0.11	-0.041	-0.072	**0.208**	**0.232**
Runtime.freeMemory()	-0.004	-0.008	0.143	**0.246**	0.007	0.059	**0.151**	0.175
Graphics2D.fill()	**0.139**	**0.171**	0.004	0.02	0.013	0.071	0.004	0.02
DriverManager.getConnection()	0.178	0.189	0.288	**0.352**	0.284	0.328	**0.301**	0.335
URL.openConnection()	-0.004	-0.004	0.408	0.476	0.357	0.379	**0.506**	**0.578**
File.toURI()	-0.004	-0.004	-0.01	-0.011	-0.02	-0.038	**0.171**	**0.171**
BufferedReader.readLine()	0.087	0.116	0.18	0.247	**0.216**	0.284	0.183	**0.304**
Average	0.143	0.154	0.25	0.301	0.159	0.218	**0.313**	**0.357**

Table 3: Comparison of UnixCoder, GraphCodeBERT, and AFGNN with statistical measures

Model Pairwise	P-Value		P-Value (BH)	
Comparisons	**RI ↓**	**MI ↓**	**RI ↓**	**MI ↓**
UnixCoder and AFRGCN	0.0022	0.0008	0.0067	0.0032
UnixCoder and AFGCN	0.8938	0.1944	0.8938	0.2121
GraphCodeBERT and AFRGCN	0.0104	0.0227	0.0179	0.0302
GraphCodeBERT and AFGCN	0.0053	0.0128	0.0129	0.0191

These differences are accompanied by large effect sizes, with Cohen's d values up to -0.86 and Cliff's δ exceeding -0.6, indicating that the improvements of AFRGCN are not only statistically significant but also practically meaningful. A p-value of less than 0.05 allows us to reject the null hypothesis, highlighting the superior performance of AFRGCN, particularly in comparison to UnixCoder and GraphCodeBERT.

On the other hand, there is no statistically significant or practically meaningful difference between UnixCoder and AFGCN, as reflected by high p-values (BH-corrected p-values of 0.8938 for RI and 0.2121 for MI) and negligible effect sizes (Cohen's d close to 0 and small Cliff's δ values). In addition, GraphCodeBERT performs better than AFGCN (with a large effect size). The reason is that AFGCN does not differentiate between FD, CD, and SE, which makes it less expressive, although AFRGCN achieves the best overall performance.

6.2 RQ2: API Misuse Detection on MUBench

We evaluate AFGNN on the MUBench[11] dataset to assess its effectiveness on real-world benchmarks. We first compare AFGNN with GraphCodeBERT, which performs best among the small LM baselines, as shown in RQ1. Table 4 summarizes the evaluation results on MUBench. We have used the RGCN variant of AFGNN (AFRGCN) as it performed the best. For the evaluation, we generate API usage embeddings for a large set of historic code examples (from the pre-training data in Section 3.2.2) for the MUBench APIs using both AFRGCN and GraphCodeBERT and then cluster them, and map the MUBench examples to these clusters to predict correct usage or potential misuse based on cluster sizes. We have used a Birch threshold of 3.0 based on our experiments and ablation studies. We experimented with multiple threshold values (e.g., 1.5, 2.0, 2.5, 3.0) and observed that increasing the threshold initially improves accuracy and precision, but values beyond 3.0 do not yield further gains. A threshold of 3.0 provided the best balance between precision, recall, and F1-score, which is why it was selected.

Table 4: Misuse detection on MUBench

Threshold	GraphCodeBERT-Birch (3.0)				AFRGCN-Birch (3.0)			
	Accuracy	Precision	Recall	F1-score	Accuracy	Precision	Recall	F1-score
5%	**0.573**	0.65	0.317	0.426	0.547	0.557	0.487	0.52
10%	**0.573**	0.636	0.341	0.444	**0.584**	0.572	0.687	0.625
15%	0.53	0.542	0.39	0.453	0.566	0.553	0.712	0.622
20%	0.518	0.521	0.439	0.476	0.534	0.523	0.837	0.644
25%	0.518	0.52	0.463	0.49	0.528	0.519	0.85	0.644
30%	0.518	0.52	0.475	0.496	0.534	0.522	0.862	**0.65**
35%	0.518	0.52	0.475	0.496	0.522	0.514	0.875	0.648
40%	0.524	0.526	0.487	**0.506**	0.522	0.514	0.875	0.648

Table 5: Performance of Misuse detectors on MUBench

Detectors	Precision	Recall	F1-Score
MuDetect [12]	33.00%	42.20%	36.96%
KGAMD [41]	60.00%	28.45%	38.59%
GraphiMuse [37]	42.00%	54.50%	47.44%
Li et al. [29]	**72.22%**	43.01%	53.91%
AFRGCN (threshold = 30%)	52.2%	**86.2%**	**65.0%**

Specifically, while the 2.5 threshold achieved an accuracy of 0.543, precision of 0.574, recall of 0.534, and F1-score of 0.553, the 3.0 threshold improved performance to an accuracy of 0.584, precision of 0.572, recall of 0.687, and an F1-score of 0.625 (see the second row in Table 4). We conducted a similar exercise for all baseline models and reported their best-performing versions.

To distinguish between large and small clusters, we have experimented with multiple thresholds as shown in Table 4. The threshold is a percentage of the total number of examples for an API, which means if there are 100 examples that are clustered, and the threshold is 10% then the clusters having at least 10 examples will be considered as large (correct usage).

As shown in Table 4, it reveals a clear trade-off between precision and recall as the threshold varies: lower thresholds (e.g., 10%) favour higher precision, while higher thresholds (e.g., 30 - 35%) improve recall. A 30% threshold provides the best balance, yielding the highest F1-score for AFRGCN. AFRGCN outperforms GraphCodeBERT across all metrics, achieving a best F1-score of 0.65, which is 11% higher than GraphCodeBERT's best F1-score of 0.506.

Table 5 presents the results reported by some of the popular misuse detectors. Using the F1-optimal operating point of 30%, AFRGCN achieves 20.6% improvement over the previous state-of-the-art by Li et al. [29] on the MUBench dataset.

6.3 RQ3: Impact of the Sequence Edges in AFG

A key novelty of our work is the AFG representation used by AFGNN to cluster API usage examples effectively. AFG introduces sequence edges (SE) that capture API call order and read-read dependencies, such as enforcing that `file.close()` follows `file.open()`, which are not represented in standard data or control dependency graphs (e.g., those used in GraphCodeBERT). We analyze the impact of SE edges on AFGNN's performance in identifying API usage patterns.

```
1 public class Example{
2   public void initDocumentCache(Book book){
3     Thread documentIndexerThread = new Thread(new
          DocumentIndexer(book),
          "DocumentIndexer");
4     documentIndexerThread.
          setPriority(Thread.MIN_PRIORITY);
5     documentIndexerThread.start();
6   } }
```

(a) `Thread.start()` **Example 1**

```
1 public class Example{
2   public void BackgroundStreamSaver(InputStream
        in,OutputStream out){
3     Thread myThread = new Thread(this,
          getClass().getName());
4     myThread.setPriority(Thread.MIN_PRIORITY);
5     myThread.start();
6   } }
```

(b) `Thread.start()` **Example 2**

Figure 10: Two semantically similar usages of the `Thread.start()` API method.

Table 6 presents results with and without SE edges. Adding SE edges significantly improves the performance of AFRGCN, yielding a 48% and 40% increase in RI and MI scores, respectively; AFGCN also shows notable gains.

Figure 10a and 10b are two similar code examples using the API method `Thread.start()` where the thread is created, priority is set to minimum, and finally, the thread is started. AFGNN keeps these two examples in the same cluster when the SE edge between Lines 4 and 5 (`Thread.setPriority()` to `Thread.start()`) is there; otherwise, it considers the two usage examples different, illustrating the effect of SE edges.

6.4 RQ4: Effect of AFG Pruning

Rather than using raw AFGs, we apply the pruning strategy described in Section 3.1.3 to remove nodes and edges unrelated to the API of interest. This step enhances API usage localization by retaining only relevant dependencies. We evaluated AFGNN's performance with and without pruning and found that, on an average, across all 21 APIs, AFRGCN achieves an RI score of 0.313 and an MI score of 0.357 with pruning, compared to 0.298 and 0.346 without. Thus, for AFRGCN, pruning of AFGs improves the effectiveness, leading to a notable increase in both RI and MI scores. We observed similar improvement for AFGCN also. The gains are smaller than those from SE edges, primarily because the API usage examples used for labelling and evaluation are typically small and focused on a single API, leaving fewer nodes and edges to prune.

7 Discussion

Threats to Validity. We evaluated clusters using APIs labeled by two expert programmers, who established labeling rules and resolved disagreements through discussion. However, a subsequent

Table 6: Performance of AFGNN with & without sequence edges. The highest MI and RI scores are highlighted in each row.

API methods	Without Sequence Edges				With Sequence Edges			
	AFGCN-Birch(1.5)		AFRGCN-Birch(2.1)		AFGCN-Birch(1.5)		AFRGCN-Birch(2.1)	
	RI score ↑	MI score ↑	RI score ↑	MI score ↑	RI score ↑	MI score ↑	RI score ↑	MI score ↑
Classloader.getResource()	0.219	0.206	0.173	0.226	0.254	0.23	**0.322**	**0.396**
Thread.start()	0.114	0.231	0.124	0.143	0.114	0.231	**0.439**	**0.472**
Statement.execute()	0.065	0.165	0.211	0.268	**0.34**	**0.422**	0.174	0.184
BufferedReader.read()	0.056	0.117	0.248	0.326	0.144	0.238	**0.412**	**0.492**
Timestamp.compareTo()	0.39	0.489	**0.492**	**0.492**	0.39	0.489	0.325	0.325
DataInputStream.readLine()	0.445	0.5	0.289	0.315	0.383	0.423	**0.41**	**0.465**
ServerSocket.bind()	0.093	0.214	**0.317**	**0.379**	0.098	0.198	0.231	0.298
ExecutorService.submit()	-0.016	-0.032	-0.014	-0.016	0.021	0.056	**0.278**	**0.306**
URI.getFragment()	0.192	0.376	0.547	0.613	0.433	0.449	**0.683**	**0.719**
Calendar.getTime()	0.023	0.081	**0.318**	0.356	0.064	0.207	0.313	**0.425**
Socket.connect()	0.034	0.143	0.465	0.533	0.198	0.412	**0.58**	**0.621**
JPanel.add()	0.048	0.14	0.107	0.227	0.055	0.122	**0.384**	**0.49**
FileChannel.read()	-0.015	-0.019	-0.014	-0.014	-0.007	-0.007	**0.142**	**0.157**
DateFormat.format()	0.031	0.074	0.102	0.119	0.042	0.096	**0.364**	**0.339**
ClassLoader.loadClass()	-0.041	-0.072	0.057	0.068	-0.041	-0.072	**0.208**	**0.232**
Runtime.freeMemory()	0.009	0.026	**0.213**	**0.282**	0.007	0.059	0.151	0.175
Graphics2D.fill()	0.006	0.026	**0.188**	**0.229**	0.013	0.071	0.004	0.02
DriverManager.getConnection()	**0.433**	**0.466**	0.366	0.389	0.284	0.328	0.301	0.335
URL.openConnection()	0.34	0.404	0.133	0.226	0.357	0.379	**0.506**	**0.578**
File.toURI()	-0.022	-0.056	-0.014	-0.015	-0.02	-0.038	**0.171**	**0.171**
BufferedReader.readLine()	0.095	0.139	0.118	0.224	**0.216**	0.284	0.183	**0.304**
Average	0.119	0.172	0.211	0.256	0.159	0.218	**0.313**	**0.357**

review by the authors, although finding most labels agreeable, identified areas for improvement, highlighting the challenge of fully eliminating human bias. With additional resources (e.g., capital and manpower), the labeling process can be extended by involving more reviewers to supervise annotations, thereby reducing disagreements and improving the reliability of the ground-truth.

AFGNN, like any data-driven technique, reflects the usage patterns present in the code it uses. Therefore, when an API evolves and its usage protocol changes, the model may continue to capture outdated patterns until newer practices become prevalent in real-world code. This temporal dependency is a general limitation of data-driven and machine-learning-based approaches, rather than one specific to AFGNN. In addition, our approach requires a sufficient number of usage examples for effective clustering. For rarely used or unknown APIs, the lack of sufficient data can limit AFGNN's applicability, a limitation shared by most LLM-based and static analysis approaches, which also rely on prior knowledge of the API's semantics and usage patterns. Thus, while AFGNN can generalize well to APIs with enough representative examples, its effectiveness may decrease for low-frequency or entirely novel APIs.

Design Decisions. We experimented with various small LMs, including CodeBERT, GraphCodeBERT, UnixCoder, and CodeT5+, as baselines. Among these, GraphCodeBERT and UnixCoder gave the best results, while CodeBERT and CodeT5+ produced significantly poorer clusters (with RI/MI scores close to 0 in most cases). GraphCodeBERT and UnixCoder performed well due to their consideration of token sequences, data flow, and AST information, emphasizing the importance of structural and semantic information for API usage tasks. Although CodeT5+ underperformed in clustering, we use it for initializing node embeddings in AFGNN due to its strong performance in this context (0.902 accuracy, compared to 0.786 of CodeBERT), with lower dimensionality (256d, compared to 768d of CodeBERT) and better computational efficiency.

To determine the optimal number of clusters, we experimented with multiple internal validation metrics like the Davies-Bouldin Index (DBI) [16], Silhouette score [43], and Calinski-Harabasz Index [15]. Similarly, for external validation, we experimented with the Rand Index (RI) [26], Mutual Information (MI) score [48], V-measure [42], and Fowlkes Mallows score [20]. The correct metrics to use depends on the domain. Based on our experiments, we chose DBI for internal validation and RI, MI scores for external validation.

8 Conclusion

Understanding API usage patterns and detecting misuse can enhance security, saving time and effort. Existing state-of-the-art models underperform in this task as they do not consider API-specific flow information. AFGNN is effective by incorporating data flow, control flow, and API call sequences in code into a novel graph representation called the API Flow Graph (AFG). It does not need any human intervention and can work with any API as it relies on vast open-source software repositories for usage examples. Additionally, for any given API name, AFGNN can recommend frequent usage patterns by analyzing clusters of historical code examples, focusing on cluster centroids, and ranking these patterns based on cluster sizes. This paper presents a detailed evaluation of AFGNN, demonstrating its superior performance in detecting potential misuse in real-world examples compared to state-of-the-art misuse detectors and small LMs.

References

[1] 2024. Supplementary Material. https://doi.org/10.5281/zenodo.15352934. Accessed: 2025-01-01.

[2] 2025. Code-Kernel dataset. https://codekernel19.github.io/demo_api.html.

[3] 2025. Example of BufferedReader API misuse. https://github.com/YuuuJeong/Algo_Study/commit/ed463554dcd240f5047bc4225fd02c419265fbe3.

[4] 2025. Huggingface Transformers Library. https://github.com/huggingface/transformers.

[5] 2025. Java-large dataset. https://s3.amazonaws.com/code2seq/datasets/java-large.tar.gz.

[6] 2025. Sklearn Metrics. https://scikit-learn.org/stable/api/sklearn.metrics.html.

[7] 2025. Stack Overflow. https://stackoverflow.com/. Accessed: 23 October 2025.

[8] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[9] Miltiadis Allamanis. 2022. Graph neural networks in program analysis. *Graph neural networks: foundations, frontiers, and applications* (2022), 483–497.

[10] Uri Alon, Shaked Brody, Omer Levy, and Eran Yahav. 2019. code2seq: Generating Sequences from Structured Representations of Code. In *International Conference on Learning Representations*. https://openreview.net/forum?id=H1gKYo09tX

[11] Sven Amann, Sarah Nadi, Hoan A Nguyen, Tien N Nguyen, and Mira Mezini. 2016. MUBench: A benchmark for API-misuse detectors. In *Proceedings of the 13th international conference on mining software repositories*. 464–467.

[12] Sven Amann, Hoan Anh Nguyen, Sarah Nadi, Tien N. Nguyen, and Mira Mezini. 2019. Investigating next steps in static API-misuse detection. In *Proceedings of the 16th International Conference on Mining Software Repositories* (Montreal, Quebec, Canada) *(MSR '19)*. IEEE Press, 265–275. doi:10.1109/MSR.2019.00053

[13] Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. 2024. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954* (2024).

[14] Dan Busbridge, Dane Sherburn, Pietro Cavallo, and Nils Y Hammerla. 2019. Relational graph attention networks. *arXiv preprint arXiv:1904.05811* (2019).

[15] T. CaliÑski and J Harabasz. 1974. A dendrite method for cluster analysis. *Communications in Statistics* 3, 1 (1974), 1–27. arXiv:https://www.tandfonline.com/doi/pdf/10.1080/03610927408827101 doi:10.1080/03610927408827101

[16] David L. Davies and Donald W. Bouldin. 1979. A Cluster Separation Measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-1, 2 (1979), 224–227. doi:10.1109/TPAMI.1979.4766909

[17] Alan L Davis and Robert M Keller. 1982. Data flow program graphs. (1982).

[18] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, Online, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139

[19] Jeanne Ferrante, Karl J Ottenstein, and Joe D Warren. 1987. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 9, 3 (1987), 319–349.

[20] E. B. Fowlkes and C. L. Mallows. 1983. A Method for Comparing Two Hierarchical Clusterings. *J. Amer. Statist. Assoc.* 78, 383 (1983), 553–569. arXiv:https://www.tandfonline.com/doi/pdf/10.1080/01621459.1983.10478008 doi:10.1080/01621459.1983.10478008

[21] Miles Frantz, Ya Xiao, Tanmoy Sarkar Pias, Na Meng, and Danfeng Yao. 2024. Methods and benchmark for detecting cryptographic api misuses in python. *IEEE Transactions on Software Engineering* 50, 5 (2024), 1118–1129.

[22] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2019. CodeKernel: A Graph Kernel Based Approach to the Selection of API Usage Examples. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 590–601. doi:10.1109/ASE.2019.00061

[23] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. doi:10.48550/ARXIV.2203.03850

[24] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366* (2020).

[25] Weihua Hu*, Bowen Liu*, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. 2020. Strategies for Pre-training Graph Neural Networks. In *International Conference on Learning Representations*. https://openreview.net/forum?id=HJlWWJSFDH

[26] Lawrence J. Hubert and Phipps Arabie. 1985. Comparing partitions. *Journal of Classification* 2 (1985), 193–218. https://api.semanticscholar.org/CorpusID:189915041

[27] Hong Jin Kang and David Lo. 2021. Active learning of discriminative subgraph patterns for api misuse detection. *IEEE Transactions on Software Engineering* 48, 8 (2021), 2761–2783.

[28] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[29] Can Li, Jingxuan Zhang, Yixuan Tang, Zhuhang Li, and Tianyue Sun. 2024. Boosting API Misuse Detection via Integrating API Constraints from Multiple Sources. In *Proceedings of the 21st International Conference on Mining Software Repositories* (Lisbon, Portugal) *(MSR '24)*. Association for Computing Machinery, New York, NY, USA, 14–26. doi:10.1145/3643991.3644904

[30] Wenqing Li, Shijie Jia, Limin Liu, Fangyu Zheng, Yuan Ma, and Jingqiang Lin. 2022. Cryptogo: Automatic detection of go cryptographic api misuses. In *Proceedings of the 38th Annual Computer Security Applications Conference*. 318–331.

[31] Xia Li, Jiajun Jiang, Samuel Benton, Yingfei Xiong, and Lingming Zhang. 2021. A Large-scale Study on API Misuses in the Wild. In *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*. 241–252. doi:10.1109/ICST49551.2021.00034

[32] Zhenmin Li and Yuanyuan Zhou. 2005. PR-Miner: automatically extracting implicit programming rules and detecting violations in large software code. *ACM SIGSOFT Software Engineering Notes* 30, 5 (2005), 306–315.

[33] Christian Lindig. 2015. Mining patterns and violations using concept analysis. In *The Art and Science of Analyzing Software Data*. Elsevier, 17–38.

[34] Yu Luo, Weifeng Xu, and Dianxiang Xu. 2022. Compact abstract graphs for detecting code vulnerability with GNN models. In *Proceedings of the 38th Annual Computer Security Applications Conference*. 497–507.

[35] Chen Lyu, Ruyun Wang, Hongyu Zhang, Hanwen Zhang, and Songlin Hu. 2021. Embedding API dependency graph for neural code generation. *Empirical Software Engineering* 26 (2021), 1–51.

[36] Wei Ma, Mengjie Zhao, Ezekiel Soremekun, Qiang Hu, Jie Zhang, Mike Papadakis, Maxime Cordy, Xiaofei Xie, and Yves Le Traon. 2022. GraphCode2Vec: Generic Code Embedding via Lexical and Program Dependence Analyses. arXiv:2112.01218 [cs.SE]

[37] Yunlong Ma, Wentong Tian, Xiang Gao, Hailong Sun, and Li Li. 2024. API Misuse Detection via Probabilistic Graphical Model. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 88–99.

[38] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781* (2013).

[39] Martin Monperrus and Mira Mezini. 2013. Detecting missing method calls as violations of the majority rule. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 22, 1 (2013), 1–25.

[40] OpenAI. 2023. ChatGPT (Mar 14 version) [Large language model]. https://chat.openai.com/chat. Accessed: 23 Oct 2025.

[41] Xiaoxue Ren, Xinyuan Ye, Zhenchang Xing, Xin Xia, Xiwei Xu, Liming Zhu, and Jianling Sun. 2020. API-Misuse Detection Driven by Fine-Grained API-Constraint Knowledge Graph. In *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 461–472.

[42] Andrew Rosenberg and Julia Hirschberg. 2007. V-Measure: A Conditional Entropy-Based External Cluster Evaluation Measure. In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, Jason Eisner (Ed.). Association for Computational Linguistics, Prague, Czech Republic, 410–420. https://aclanthology.org/D07-1043

[43] Peter J. Rousseeuw. 1987. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *J. Comput. Appl. Math.* 20 (1987), 53–65. doi:10.1016/0377-0427(87)90125-7

[44] Christopher Scaffidi. 2006. Why are APIs difficult to learn and use? *XRDS* 12, 4 (Aug. 2006), 4. doi:10.1145/1144359.1144363

[45] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2020. Mpnet: Masked and permuted pre-training for language understanding. *Advances in neural information processing systems* 33 (2020), 16857–16867.

[46] Thiviyan Thanapalasingam, Lucas van Berkel, Peter Bloem, and Paul Groth. 2022. Relational graph convolutional networks: a closer look. *PeerJ Computer Science* 8 (2022), e1073.

[47] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).

[48] Nguyen Xuan Vinh, Julien Epps, and James Bailey. 2010. Information Theoretic Measures for Clusterings Comparison: Variants, Properties, Normalization and Correction for Chance. *J. Mach. Learn. Res.* 11 (dec 2010), 2837–2854.

[49] Jingjing Wang, Minhuan Huang, Xiang Li, Qianjin Du, Wei Kong, Huan Deng, Xiaohui Kuang, et al. 2024. Suitable is the Best: Task-Oriented Knowledge Fusion in Vulnerability Detection. *Advances in Neural Information Processing Systems* 37 (2024), 121131–121155.

[50] Xiaoke Wang and Lei Zhao. 2023. Apicad: Augmenting api misuse detection through specifications from code and documents. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 245–256.

[51] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi Bui, Junnan Li, and Steven Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in*

Natural Language Processing, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 1069–1088. doi:10.18653/v1/2023.emnlp-main.68

[52] Moshi Wei, Nima Shiri Harzevili, YueKai Huang, Jinqiu Yang, Junjie Wang, and Song Wang. 2024. Demystifying and detecting misuses of deep learning apis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.

[53] Ming Wen, Yepang Liu, Rongxin Wu, Xuan Xie, Shing-Chi Cheung, and Zhendong Su. 2019. Exposing library API misuses via mutation analysis. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 866–877.

[54] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE symposium on security and privacy*. IEEE, 590–604.

[55] Tian Zhang, Raghu Ramakrishnan, and Miron Livny. 1997. BIRCH: A new data clustering algorithm and its applications. *Data mining and knowledge discovery* 1 (1997), 141–182.

[56] Tianyi Zhang, Ganesha Upadhyaya, Anastasia Reinhardt, Hridesh Rajan, and Miryung Kim. 2018. Are code examples on an online q&a forum reliable? a study of api misuse on stack overflow. In *Proceedings of the 40th international conference on software engineering*. 886–896.

[57] Li Zhong and Zilong Wang. 2024. Can LLM Replace Stack Overflow? A Study on Robustness and Reliability of Large Language Model Code Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 21841–21849.